The Journal of Open Source Software

WattAMeter: A Python Toolkit for Time-Series CPU and GPU Power Monitoring in HPC

Weslley da Silva Pereira[1] **and Struan Clark**[1]

1 National Laboratory of the Rockies, United States

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor:

Submitted: 13 April 2026
Published: unpublished

Summary

WattAMeter is an open-source Python toolkit for CPU and GPU power telemetry which provides synchronized time-series data during computational workloads (da Silva Pereira & Clark, 2026). It targets users who need consistent measurement workflows in scientific and engineering environments without building custom instrumentation pipelines for each project.

The software provides both a Python API and a command-line interface (CLI). At runtime, it periodically samples metrics from Linux CPU powercap/RAPL interfaces and NVIDIA GPUs through NVML, then records structured logs for downstream analysis (Linux Kernel Developers, 2026; NVIDIA Corporation, 2026). Optional MQTT publishing enables live integration with dashboards and external monitoring services while retaining local file output as the default persistent record (Standard, 2019; WattAMeter Developers, 2026).

WattAMeter is designed for high-performance computing (HPC) workflows where jobs run under schedulers, may span multiple nodes, and require measurement methods that are lightweight, portable, and non-intrusive. In addition to standalone package usage, WattAMeter is deployed as a loadable module in the National Laboratory of the Rockies' (NLR) HPC environment, where users can start and stop tracking sessions inside SLURM jobs with documented operational guidance (National Laboratory of the Rockies, 2026).

Statement of need

Energy and power are now central evaluation dimensions in computational research, especially in HPC systems where short-lived fluctuations can matter as much as totals. Users increasingly need time-resolved answers to questions such as: when power or temperature spikes occur and how to mitigate them before they stress equipment, whether short transients correlate with instability or throttling, and whether power profile history can be used to forecast near-term demand for scheduler- or runtime-level decisions. Many existing tools focus on aggregate reporting, while researchers and operators still face practical gaps when they need scheduler-aware time-series telemetry that can be embedded directly into experiments and operations workflows.

WattAMeter addresses this need by combining heterogeneous hardware telemetry into one Python-native interface and one CLI workflow. It supports periodic collection of CPU and GPU metrics in a common time-series format and can be launched as part of an experiment script or from SLURM job scripts. This lowers friction for users who need to run repeated measurements across different machines and software versions.

The package emphasizes repeatability and operational simplicity:

- deterministic sampling intervals,
- configurable write frequency while guaranteeing a final write on shutdown,

- parity between Python API and CLI interfaces,
- optional real-time publication over MQTT, and
- shell utilities that integrate with SLURM jobs.

The resulting workflow reduces the overhead for experiment instrumentation and supports comparative studies across nodes, job configurations, and application versions.

State of the field

The ecosystem includes mature components for device telemetry and energy accounting. NVIDIA tooling and NVML bindings expose GPU counters (NVIDIA Corporation, 2026), and Linux powercap/RAPL interfaces expose CPU-domain energy counters (Linux Kernel Developers, 2026). At the tooling level, Scaphandre provides host-level energy metrology with exporter integrations for time-series monitoring stacks (hubblo-org contributors, 2026), Kepler provides Kubernetes-oriented power telemetry and attribution at node/pod/container scopes (sustainable-computing-io contributors, 2026), and Zeus provides energy monitoring for deep-learning workloads with support for measurement and optimization workflows (You et al., 2023). CodeCarbon remains a widely used reference for process-level energy and carbon accounting (Courty et al., 2024). pyJoules is another well-designed Python energy-measurement library, particularly strong for process/function or code-region instrumentation workflows (Belgaid et al., 2019).

Compared with this landscape, WattAMeter is explicitly Python-native for both in-code instrumentation and CLI operation, and it is designed around scheduler-managed HPC usage, including multi-node SLURM workflows. Published documentation for the tools cited above generally emphasizes exporters, dashboards, carbon accounting, or container orchestration rather than multi-node SLURM orchestration. Thus, WattAMeter is complementary to these tools: it focuses on synchronized CPU+GPU time-series telemetry in scheduler-managed HPC workflows, from which aggregate metrics such as total energy can be inferred by integration.

In terms of extensibility, Scaphandre and Kepler emphasize integration through observability/export pipelines and deployment patterns, while Zeus emphasizes programmable ML-energy measurement and optimization workflows. CodeCarbon provides strong extensibility for outputs and integrations (for example, custom output handlers and multiple built-in sinks), while hardware-tracker extension is more tightly coupled to its internal emissions and resource-tracking stack. pyJoules exposes clear device and handler abstractions, but its common usage remains centered on function- or scope-level traces. WattAMeter prioritizes backend extensibility through reader abstractions, allowing new hardware backends without changing the user-facing tracking interface in Python scripts or SLURM wrappers.

The WattAMeter contribution is a workflow-oriented collection layer that unifies CPU and GPU telemetry in one configuration model, with emphasis on scheduler-managed HPC operation. Compared with assembling custom scripts around individual low-level interfaces, WattAMeter provides:

- a unified entry point for multi-metric tracking,
- Python-native in-code instrumentation,
- direct SLURM integration through `start_wattameter` and `stop_wattameter` CLI utilities,
- multi-node session orchestration for repeated start/stop tracking within job allocations,
- file outputs for direct ingestion in analysis workflows, and
- optional MQTT streaming for live monitoring and integration with external services.

Software design

WattAMeter follows a modular reader and tracker architecture (da Silva Pereira & Clark, 2026). Reader classes encapsulate device-specific access (for example RAPL and NVML), while tracker

classes manage periodic polling, buffering, and persistence. In practice, this means users can change metrics and sampling settings without modifying the underlying collection logic.

The same architecture is intentionally designed for straightforward extension to new backends. As a concrete example, the pull request adding an `AMDSMIReader` for AMD GPU monitoring extends the existing reader model without requiring changes to the tracker abstraction (da Silva Pereira, 2026).

The core tracker loop is thread-based and uses fixed sampling intervals (`dt_read`) with configurable write cadence (`freq_write`). On shutdown, trackers perform a final read/write cycle to reduce end-of-run data loss in short jobs or interrupted sessions. The implementation also supports context-manager usage and signal-aware command-line execution, which maps well to batch-system lifecycle control.

Two design choices are central for research use.

1. The software prioritizes periodic sampling over event-driven callbacks to produce uniformly sampled series that are easier to compare under controlled conditions. The tradeoff is that transients shorter than the selected interval may be missed. Each sample stores both a timestamp and a read-duration field, enabling users to assess telemetry overhead and timing fidelity in high-frequency collection.
2. WattAMeter uses dual-output semantics: local files are always first-class, and MQTT publication is optional. This ensures robust baseline operation even when network services are unavailable, while still supporting live observability when needed.

For scheduler workflows, shell utilities wrap `srun` orchestration to start and stop tracking sessions across allocated nodes, including support for multiple sequential sessions in one job allocation (da Silva Pereira & Clark, 2026; National Laboratory of the Rockies, 2026). Tracking sessions do not block resources and can be started and stopped independently of the main workload. The implementation includes benchmark utilities for sampling-frequency characterization and runtime overhead estimation, plus post-processing helpers for parsing and aligning log files.

Research impact statement

WattAMeter has practical impact for teams running experiments on mixed CPU and GPU nodes, because it packages data collection and operational orchestration into one reusable workflow. Instead of writing cluster-specific wrappers for each study, users can adopt a common interface for repeated measurement studies.

This impact is already visible in two concrete outputs. WattAMeter was used to generate the publicly released "Dataset of Generative AI Workload Power Profiles" in the NLR Data Catalog (Vercellino et al., 2026a). The same dataset, together with WattAMeter post-processing functionality, was then used in the study "Measurement of Generative AI Workload Power Profiles for Whole-Facility Data Center Infrastructure Planning" (Vercellino et al., 2026b).

Additional impact is evidenced by the deployment of WattAMeter as an NLR HPC module, with documented `module load wattameter` usage and scheduler guidance for production SLURM jobs (National Laboratory of the Rockies, 2026). The documentation includes concrete operational practices, including paired start/stop commands, interpretation of node-level CPU counters, and use of exclusive allocations for CPU analyses, indicating active use beyond a purely aspirational design.

The project is structured for ongoing reuse:

- permissive BSD-3-Clause licensing,
- command-line and API documentation with runnable examples,

- automated tests spanning readers, trackers, CLI parsing, shell utilities, MQTT, and post-processing,
- benchmark tooling for overhead characterization, and
- integration paths for both offline analysis and live monitoring.

These attributes make WattAMeter suitable as measurement infrastructure for performance studies, energy-aware benchmarking, and operations-focused workload analysis where transparent and repeatable telemetry collection is required.

AI usage disclosure

This manuscript draft was prepared with assistance from GPT-5.3-Codex. The authors are responsible for all technical claims and manually reviewed and edited the text for accuracy and clarity.

Acknowledgements

This work was authored by the National Laboratory of the Rockies (NLR) for the U.S. Department of Energy (DOE), operated under Contract No. DE-AC36-08GO28308. Funding was provided by the NLR. The views expressed in the article do not necessarily represent the views of the DOE or the U.S. Government. The U.S. Government retains and the publisher, by accepting the article for publication, acknowledges that the U.S. Government retains a nonexclusive, paid-up, irrevocable, worldwide license to publish or reproduce the published form of this work, or allow others to do so, for U.S. Government purposes.

We thank Matt Selensky (NLR) for maintaining the WattAMeter module on the NLR HPC system and supporting operational deployment.

References

Belgaid, M. C., Rouvoy, R., & Seinturier, L. (2019). *Pyjoules: Python library that measures python code snippets*. PyPI: https://pypi.org/project/pyJoules/. https://pyjoules.readthedocs.io

Courty, B., Schmidt, V., Luccioni, S., Goyal-Kamal, Coutarel, M., Feld, B., Lecourt, J., Connell, L., Saboni, A., Inimaz, supatomic, Leval, M., Blanche, L., Cruveiller, A., ouminasara, Zhao, F., Joshi, A., Bogroff, A., Lavoreille, H. de, … MinervaBooks. (2024). *mlco2/codecarbon: v2.4.1* (Version v2.4.1). Zenodo. https://doi.org/10.5281/zenodo.11171501

da Silva Pereira, W. (2026). *Add AMDSMIReader for monitoring AMD GPUs (pull request #9)*. https://github.com/NatLabRockies/WattAMeter/pull/9.

da Silva Pereira, W., & Clark, S. (2026). *WattAMeter*. https://github.com/NatLabRockies/WattAMeter

hubblo-org contributors. (2026). *Scaphandre: Energy consumption metrology agent*. https://github.com/hubblo-org/scaphandre

Linux Kernel Developers. (2026). *Power capping framework*. https://www.kernel.org/doc/html/latest/power/powercap/powercap.html.

National Laboratory of the Rockies. (2026). *WattAMeter: National laboratory of the rockies HPC documentation*. https://natlabrockies.github.io/HPC/Documentation/Development/Performance_Tools/WattAMeter/.

NVIDIA Corporation. (2026). *NVIDIA management library (NVML) API reference*. https://docs.nvidia.com/deploy/nvml-api/.

Standard, O. (2019). *MQTT version 5.0*. https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html.

sustainable-computing-io contributors. (2026). *Kepler: Kubernetes-based efficient power level exporter*. https://github.com/sustainable-computing-io/kepler

Vercellino, R., Willard, J., Campos, G., da Silva Pereira, W., Hull, O., Selensky, M., & Mueller, J. (2026a). *Dataset of generative AI workload power profiles*. NLR Data Catalog, National Laboratory of the Rockies, Golden, CO. https://doi.org/10.7799/3025227

Vercellino, R., Willard, J., Campos, G., da Silva Pereira, W., Hull, O., Selensky, M., & Mueller, J. (2026b). Measurement of generative AI workload power profiles for whole-facility data center infrastructure planning. *arXiv Preprint arXiv:2604.07345*. https://doi.org/10.48550/arXiv.2604.07345

WattAMeter Developers. (2026). *MQTT publishing in WattAMeter*. https://natlabrockies.github.io/WattAMeter/main/mqtt_usage.html.

You, J., Chung, J.-W., & Chowdhury, M. (2023). Zeus: Understanding and optimizing GPU energy consumption of DNN training. *USENIX NSDI*.